

Computer Use Agents.

screenshot → decide → act → repeat.

"File an expense: United Airlines, \$450, Travel. Submit."

- Trivial for you.
- Six steps for a model.
- Today: build the thing that does it.

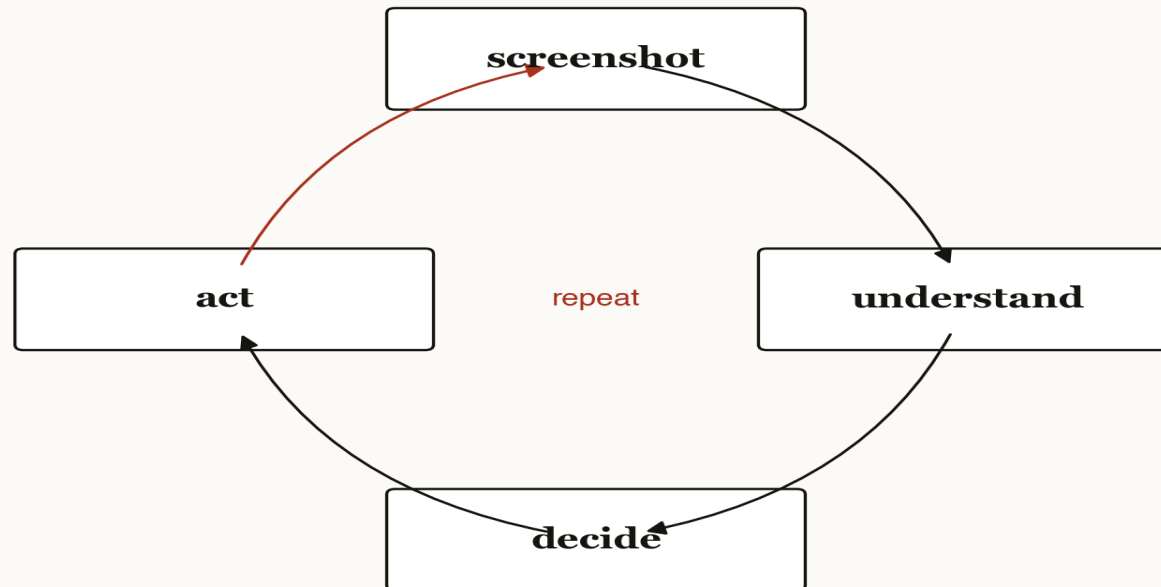
What the model sees.

- One image. 1024×768 .
 - No DOM. No element IDs. No API.
 - Pixels in $\rightarrow$ actions out.
-

The universal pattern.

The universal pattern

GPT-4V-Act, 2023 — and every agent since.



screenshot → understand → decide → act → repeat.

Stage 2 — we build it by hand.

- A general VLM over OpenRouter.
-
- We invent the action JSON.
-
- We parse it. We execute it.

It misses.

It misses

Loop: correct. Grounding: wrong.

New Expense

Vendor

United Airlines

Amount

\$450.00

predicted

Category

Travel

Submit

truth

Loop: correct. Grounding: wrong.

Referring expression → precise location, well enough to act.

— "the Submit button" → (743, 373).

— Perception ≠ grounding.

— Errors compound across steps.

Grounding is the through-line.

2016 *Universe — too early*

2023 *GPT-4V-Act · Set-of-Mark*

2024 *UFO · OmniParser*

2024 *Claude computer use*

2025 *Operator · UI-TARS*

Set-of-Mark.

Set-of-Mark

Pixel regression becomes multiple choice.

New Expense

1 Vendor
United Airlines

2 Amount
\$450.00

3 Category
Travel

4 Submit

Pixel regression → multiple choice. The detector is the grounding.

Where the detector lives.

Out of model

OmniParser · the a11y tree

Into model

UI-TARS · Claude computer use

The frontier

went "into the model"

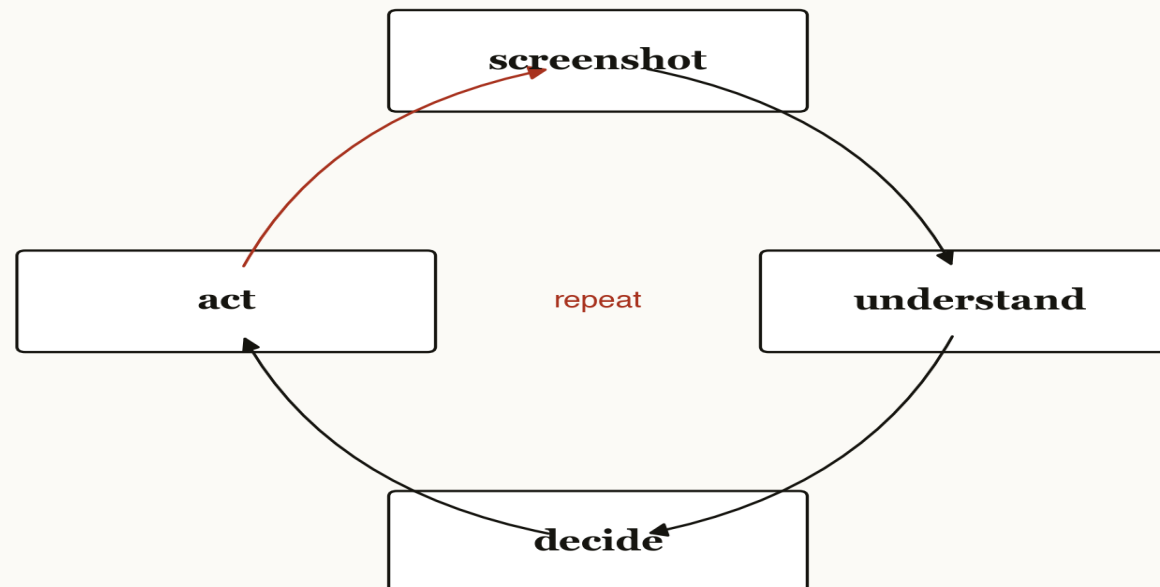
Stage 4 — swap in the native tool.

- `computer_20251124.`
- Grounding trained in.
- Fixed action space — screenshot / click / type / scroll / zoom.
- A documented agent loop.

Same browser. Same task.

The universal pattern

GPT-4V-Act, 2023 — and every agent since.



Only difference: grounding moved into the model — plus self-verification.

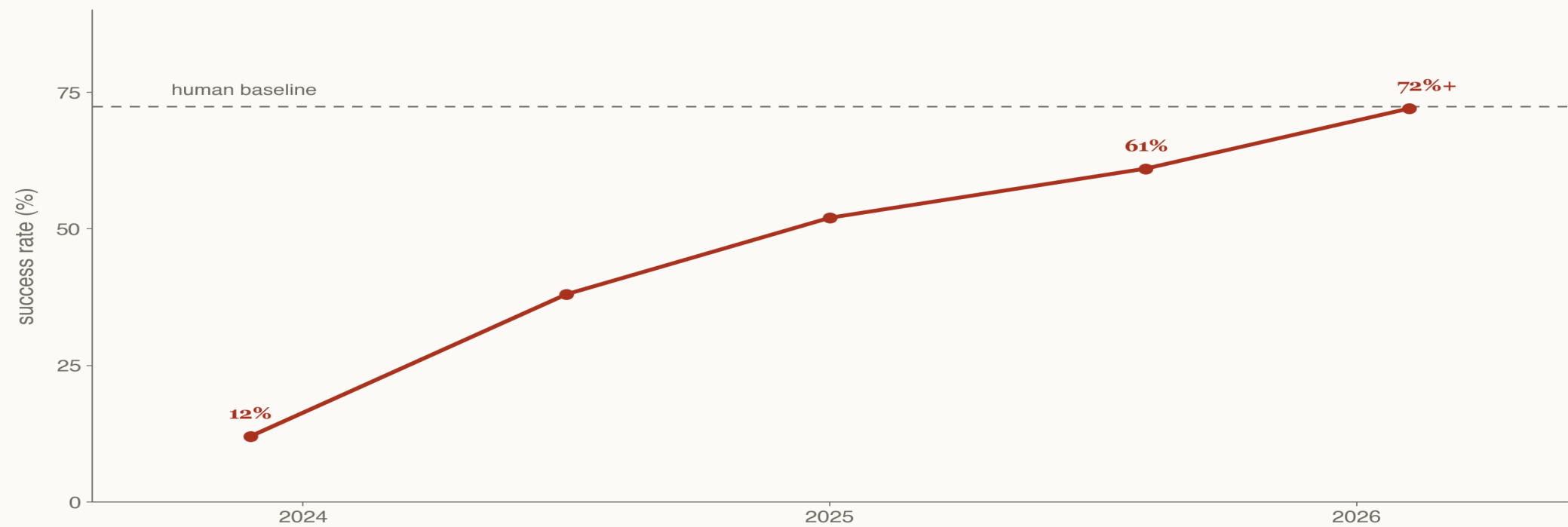
The vendor's own guidance.

- Text before screenshot → better clicks.
- zoom for small targets.
- A self-check prompt cuts "assumed it worked."
- Isolate — treat prompt injection as a given.

OSWorld.

OSWorld — success rate over time

369 real desktop tasks · execution-graded · no partial credit.



12% → 38% → 61% → 72%+ in about two years.

Read the footnote.

- OSWorld vs. OSWorld-Verified.
 - pass@1 vs. pass@10.
 - Step budget · vendor self-report.
 - Skepticism is the skill.
-

Open models.

UI-TARS

1.5 → 2 — screenshot-only, RL

UGround

a learned detector

Qwen3-VL

top open, ~66%

OmniParser

the parser

Is it even a GUI problem?

- CoAct-1 — code-as-action (hybrid).
-
- CLI-as-computer-use (geohot / Rauch).
-
- The ceiling: not everything is text.

Cua-Bench / KiCad.

- 25 expert EDA tasks.
- Best frontier agent: 6 / 25.
- Fails: budget + hallucinated screen state.
- Stage 3 — alive at the frontier.

The synthesis.

Code when efficient.

Vision when you must.

Know which is which.

Building for real.

- Isolated VM, least privilege.
- Human-in-the-loop for consequential actions.
- Log every action + screenshot.
- Cua · a reference impl · browser-use.

One frame.

	HAND-ROLLED	NATIVE TOOL
Grounding	<i>misses</i>	lands
Actions	<i>invented JSON</i>	tool schema
Loop	<i>improvised</i>	contract
Verify	<i>none</i>	self-check

The API is shaped by the pain of Stage 3.